

УДК Lorem ipsum dolor sit amet

Операционная семантика выражений в языке Go на языке ABML

Харьков А.А. (Студент ММФ НГУ)

Ануреев И.С. (Институт систем информатики СО РАН)

В данной статье представлено формальное описание операционной семантики выражений языка программирования Go с использованием языка спецификаций ABML. Рассматривается фрагмент языка Go, не включающий в себя работу с многопоточностью и соответствующими конструкциями (горутины, каналы, операторы `go` и `select`). Предложенная онтологическая модель позволяет строго задать семантику базовых конструкций языка и может служить основой для разработки инструментов верификации программ.

Ключевые слова: операционная семантика, язык Go, язык ABML, формальные методы, онтологическое моделирование.

1. Введение

Формальное описание семантики языков программирования является фундаментальной задачей, лежащей в основе разработки компиляторов, статических анализаторов и инструментов верификации программ. Традиционные подходы к спецификации семантики, основанные на естественном языке, часто страдают от неоднозначности и неполноты, что затрудняет создание формально верифицируемых инструментов.

В данной работе рассматривается фрагмент языка программирования Go и его операционная семантика, формализованная с помощью языка спецификаций ABML (Attribute-Based Modeling Language), который является лексическим расширением диалекта Common Lisp — SBCL. Выбор языка Go обусловлен его возрастающей популярностью в промышленной разработке, особенно в области системного программирования. Язык ABML, в свою очередь, предоставляет средства для описания онтологий предметных областей и формализации правил перехода между состояниями.

2. Онтология фрагмента языка Go

Построение формальной модели семантики языка программирования требует предварительного создания онтологии — системы понятий, описывающих синтаксические конструкции и семантические сущности языка. В данном разделе на языке ABML вводятся типы объектов, соответствующие элементам фрагмента языка Go.

Классификация и порядок изложения опираются на официальную спецификацию языка Go. Большинство наименований моделируемых объектов соответствуют их названиям в спецификации. Однако, часть конструкций языка или их составляющих не имеет уникальных наименований, или их названия связаны с синтаксисом, что не позволяет достаточно точно определять их смысл в различных контекстах.

2.1. Имена

Все имена в программе на Go моделируются как строки. Вводятся следующие типы имен:

```
(typedef "identifier"      (uniont string))
(typedef "constant name"  "identifier")
(typedef "variable name"  "identifier")
(typedef "field name"     "identifier")
(typedef "function name"  "identifier")
(typedef "parameter name" "identifier")
(typedef "method name"    "identifier")
(typedef "label name"     "identifier")
(typedef "type name"      "identifier")
```

2.2. Типы данных

Система типов языка Go разделяется на базовые (простые) и составные типы.

```
(typedef "type" (uniont "primitive type" "composite type"))
```

2.2.1. Базовые типы

Базовые типы включают логический, строковый и числовые.

```
(typedef "primitive type" (uniont "bool type" "string type"
                                   "numeric type"))

(cot "bool type")
(cot "string type")
```

Числовые типы детализируются. Целые типы разделяются на знаковые и беззнаковые с указанием разрядности. Вещественные и комплексные типы также имеют разрядность.

```
(typedef "numeric type" (uniont "int type" "float type" "complex
                                type"))

(typedef "int type"      (uniont "signed int type" "unsigned int type"))
(cot "signed int type"   :at "bit size" (enumt 8 16 32 64))
(cot "unsigned int type" :at "bit size" (enumt 8 16 32 64))
(cot "float type"       :av "bit size" (enumt 32 64))
(cot "complex type"     :av "bit size" (enumt 64 128))
```

2.2.2. Составные типы

Тип составных объектов включает в себя типы массивов, срезов, структур, указателей, функций, методов, интерфейсов, карт и каналов.

```
(typedef "composite type"
  (uniont "array type" "slice type" "struct type"
          "pointer type" "function type" "method type"
          "interface type" "map type" "channel type"))
```

Тип массивов содержит информацию о типе элементов и фиксированной длине. Срезы — только тип элементов, поскольку длина среза определяется динамически.

```
(cot "array type" :at "elem type" "type" :at "len" nat)
(cot "slice type" :at "elem type" "type")
```

Структура определяется набором полей. Атрибут `ordered` используется на этапе статического анализа для поддержки декларации экземпляра структуры с позиционным заданием значений полей.

```
(cot "struct type" :at "fields" (cot :amap "field name" "type")
    :at "ordered" (listt "field name"))
```

Тип указателей хранит информацию о типе, на который он указывает.

```
(cot "pointer type" :at "type" "type")
```

Функциональный тип описывает сигнатуру функции. Имена параметров опущены, так как при проверке типов важны только соответствующие типы. Вариативный параметр, позволяющий функции принимать произвольной число аргументов (задается многоточием), моделируется отдельным атрибутом.

```
(cot "function type"
    :at "param types" (listt "type")
    :at "variadic type" "type"
    :at "result types" (listt "type"))
```

Тип методов содержит получателя (receiver) и используется, в том числе, при определении интерфейсов.

```
(cot "method type"
    :at "receiver type" "type"
    :at "param types" (listt "type")
    :at "variadic type" "type"
    :at "result types" (listt "type"))
```

Тип интерфейсов определяется как набор реализуемых методов с их сигнатурами.

```
(cot "interface type"
    :at "methods" (cot :amap "method name" "method type"))
```

Тип карт задается типами ключа и значения.

```
(cot "map type" :at "key type" "type" :at "elem type" "type")
```

Канал имеет тип передаваемых элементов. Направление не моделируется, так как для корректных программ оно не влияет на семантику.

```
(cot "channel type" :at "elem type" "type")
```

2.3. Ячейки памяти и значения

В данной работе модель памяти представлена типизированными значениями. То есть каждое значение имеет собственный тип, известный еще до этапа выполнения программы.

Ячейка представляет собой изменяемый контейнер, хранящий значение. Переменные связываются с ячейками, а не непосредственно со значениями. Тип ячейки указывает на тип хранимого значения.

```
(mot "cell" :at "value" "Go value" :at "type" "type")
(typedef "Go value" (uniont "untyped constant" "typed primitive"
                           "composite value"))
```

Нетипизированные константы существуют только на этапе статического анализа. До начала выполнения программы в операционной семантике они преобразуются в типизированные значения.

```
(typedef "untyped constant" (uniont bool int real string complex))
```

Типизированные примитивы моделируют простейшие значения, то есть для хранения которых не используются ячейки памяти, с их статическими типами. Такими значениями являются строки и числа, в том числе вещественные и комплексные.

```
(mot "typed primitive"
      :at "type" "type"
      :at "value" (uniont "nil" "untyped constant"))
```

Nil-значение, что встречается в предыдущей конструкции, является значением сложных типов при инициализации. Стоит заметить, что nil-ы разных типов различаются.

```
(cot "nil")
```

Составные значения моделируют структуры данных языка Go.

```
(typedef "composite value"
  (uniont "array value" "slice value" "struct value"
    "map value" "interface value" "method value"
    "function value" "cell" "channel value"))
```

Массив хранит элементы в виде списка ячеек, что обеспечивает возможность изменения элементов. Однако, длина массива определяется собственным типом, и изменяться не может.

```
(mot "array value"
  :at "type"      "array type"
  :at "elements" (listt "cell"))
```

Срез является представлением фрагмента массива. Несколько срезов могут разделять один и тот же массив.

```
(mot "slice value"
  :at "type"      "slice type"
  :at "array"      "array value"
  :at "offset"     nat
  :at "length"     nat
  :at "capacity"  nat)
```

Структура хранит поля как отображение имен на ячейки.

```
(mot "struct value"
  :at "type"      "struct type"
  :at "fields" (mot :amap "field name" "cell"))
```

Карта хранит записи как отображение ключей на ячейки значений.

```
(mot "map value"
  :at "type"      "map type"
  :at "entries" (mot :amap "Go value" "cell"))
```

Канал представляет собой FIFO-очередь с буфером и очередями ожидающих горутин (моделируемых ячейками).

```
(mot "channel value"
  :at "type"      "channel type"
  :at "buffer"    (listt "Go value")
  :at "send queue" (listt "cell")
  :at "receive queue" (listt "cell")
  :at "closed"    bool)
```

Функция хранит тип, сигнатуру (с именами параметров), тело и замыкание — отображение захваченных переменных на ячейки.

```
(mot "function value"
  :at "type"      "function type"
  :at "signature" "function signature"
  :at "body"      "block"
  :at "closure"   (mot :amap "variable name" "cell"))
```

Сигнатура функции содержит список деклараций параметров, вариативный параметр (если есть) и результаты (могут быть безымянными — только типы, или именованными).

```
(mot "function signature"
  :at "parameters" (listt "param decl")
  :at "variadic param" "param decl"
  :at "results" (uniont (listt "type") (listt "param decl")))

(mot "param decl" :at "type" "type" :at "name" "parameter name")
```

Метод аналогичен функции, но имеет отдельный получатель. Сигнатура метода включает получатель как отдельную декларацию параметра.

```
(mot "method value"
  :at "type"      "method type"
  :at "signature" "method signature"
  :at "body"      "block"
  :at "closure"   (mot :amap "variable name" "cell"))

(mot "method signature"
  :at "receiver"      "param decl"
  :at "parameters"    (listt "param decl")
  :at "variadic param" "param decl"
  :at "results"       (listt "param decl"))
```

Интерфейс хранит статический тип и значение. Динамический тип содержится внутри значения.

```
(mot "interface value"
  :at "type" "interface type"
  :at "value" "typed value")
```

2.4. Литералы

Литералы представляют собой значения, которые могут быть непосредственно записаны в исходном коде. Общий тип литерала объединяет типизированные константы и составные литералы.

```
(typedef "literal"
  (uniont "untyped constant" "numeric lit"
    "composite lit" "function lit" "method lit"))
```

Числовые литералы используются для задания чисел с помощью различных представлений.

```
(mot "numeric lit" :at "value" string)
```

Литералы для составных значений: массив, срез, структура или карта.

```
(typedef "composite lit" (uniont "array lit" "slice lit"
                                   "struct lit" "map lit"))
```

Литералы массивов и срезов могут задаваться с явным указанием индекса для каждого значения. Каждый такой элемент представлен парой `index & elem`.

```
(mot "index & elem" :at "index" nat :at "value" "expression")

(mot "array lit"
     :at "type"      "array type"
     :at "elements" (listt "index & elem"))

(mot "slice lit"
     :at "type"      "slice type"
     :at "elements" (listt "index & elem"))
```

Литералы структур могут объявляться с явным указанием имени поля для каждого значения. Каждый такой элемент представлен парой `field & value`.

```
(mot "field & value" :at "field" "field name" :at "value" "expression")

(mot "struct lit"
     :at "type"      "struct type"
     :at "fields" (listt "field & value"))
```

Литералы карт объявляются явным перечислением пар ключ-значение.

```
(mot "key & elem" :at "key" "expression" :at "value" "expression")

(mot "map lit"
```

```

:at "type"      "map type"
:at "elements" (listt "key & elem"))

```

Функциональные литералы (анонимные функции) хранят тип, сигнатуру и тело.

```

(mot "function lit "
  :at "type"      "function type"
  :at "signature" "function signature"
  :at "body"      "block")

(mot "method lit "
  :at "type"      "method type"
  :at "signature" "method signature"
  :at "body"      "block")

```

2.5. Выражения

Выражения в языке Go — это конструкции, которые после вычисления возвращают значение. Все выражения содержат атрибут `type`, значение которого вычисляется на этапе статического анализа.

```

(typedef "expression"
  (uniont "literal" "variable ref" "(1)"
    "conversion" "method expr" "selector expr" "index expr"
    "slice expr" "type assertion" "<-1" "function call"
    "unary expression" "binary expression"))

```

Ссылка на переменную представляет использование уже объявленной переменной.

```

(mot "variable ref" :at "name" "variable name" :at "type" "type")

```

Выражение в скобках влияет только на порядок вычисления.

```

(mot "(1)" :at 1 "expression" :at "type" "type")

```

Приведение типов позволяет явно преобразовывать значения.

```
(mot "conversion" :at "type" "type" :at "value" "expression")
```

Метод-выражение возвращает функцию с получателем в качестве первого параметра.

```
(mot "method expr" :at "receiver type" "type"  
      :at "name" "method name"  
      :at "type" "type")
```

Выбор поля или метода по экземпляру.

```
(mot "selector expr" :at "receiver" "expression"  
      :at "name" "identifier"  
      :at "type" "type")
```

Индексирование применимо к массивам, срезам, картам и строкам.

```
(mot "index expr" :at "indexable" "expression"  
      :at "index" "expression"  
      :at "type" "type")
```

Выражение среза создает новый срез из существующего массива, строки или среза. Параметры `low`, `high` и `max` задают границы. Последний из них может отсутствовать.

```
(mot "slice expr"  
      :at "sequence" "expression"  
      :at "low"      "expression"  
      :at "high"     "expression"  
      :at "max"      "expression"  
      :at "type"     "slice type")
```

Проверка типа применяется к интерфейсным значениям.

```
(mot "type assertion" :at "interface" "expression" :at "type" "type")
```

Операция чтения из канала.

```
(mot "<-1" :at 1 "expression" :at "type" "type")
```

Вызов функции и метода.

```
(mot "function call" :at "function" "expression"
      :at "arguments" (listt "expression")
      :at "type" "type")

(mot "method call" :at "receiver" "expression"
      :at "method" "method name"
      :at "arguments" (listt "expression")
      :at "type" "type")
```

Унарные выражения используют цифровую нотацию для обозначения операнда.

```
(typedef "unary expression" (uniont "+1" "-1" "^1" "!1" "*1" "&1"))

(mot "+1" :at 1 "expression" :at "type" "type")
(mot "-1" :at 1 "expression" :at "type" "type")
(mot "^1" :at 1 "expression" :at "type" "type")
(mot "!1" :at 1 "expression" :at "type" "type")
(mot "*1" :at 1 "expression" :at "type" "type")
(mot "&1" :at 1 "expression" :at "type" "type")
```

Бинарные выражения также используют цифровую нотацию: **1** — левый операнд, **2** — правый.

```
(typedef "binary expression"
  (uniont "1||2" "1&&2" "1+2" "1-2" "1*2" "1/2" "1%2"
    "1<<2" "1>>2" "1&2" "1&^2" "1|2" "1^2"
    "1==2" "1!=2" "1<2" "1<=2" "1>2" "1>=2"))
```

2.6. Блоки

Блоки определяют области видимости переменных и меток. В языке Go блок представляет собой последовательность операторов, заключенную в фигурные скобки.

```
(mot "block"
  :at "statements" (listt "statement")
  ;; semantic attributes
  :at "label positions" (mot :amap "label name" nat)
  :at "all variables" (listt "variable name")
  :at "decl variables" (listt "variable name")
  :at "variable cells" (mot :amap "variable name" "cell"))
```

Атрибут `statements` содержит список операторов блока. Атрибут `label positions` предоставляет меткам их позиции в блоке. Атрибут `variable cells` сохраняет для каждой переменной ее ячейку памяти до входа в блок, что позволяет корректно обрабатывать затенение имен переменных.

2.7. Декларации

Декларации вводят новые объекты в программу.

```
(typedef "declaration" (uniont "type decl" "const decl" "var decl"
  "function decl" "method decl"))
```

Декларация типа связывает имя с определением типа.

```
(mot "type decl" :at "name" "type name" :at "type" "type")
```

Декларация констант и переменных может содержать явное указание типов (опционально) и значения.

```
(mot "const decl"
  :at "names" (listt "constant name")
  :at "types" (listt "type"))
```

```

      :at "values" (listt "expression"))

(mot "var decl"
      :at "names" (listt "variable name")
      :at "types" (listt "type")
      :at "values" (listt "expression"))

```

Декларация функций и методов.

```

(mot "function decl" :at "name" "function name"
      :at "value" "function lit")

(mot "method decl" :at "name" "method name"
      :at "value" "method lit")

```

2.8. Операторы

Операторы в языке Go представляют собой конструкции, которые изменяют состояние программы, но не возвращают значений.

```

(typedef "statement"
  (uniont "declaration" "label" "empty stmt" "1++" "1—"
    "assignment stmt" "if stmt" "switch stmt" "for stmt"
    "return stmt" "break stmt" "continue stmt" "goto stmt"
    "defer stmt" "go stmt" "1<-2" "fallthrough" "select stmt"))

```

Помеченный оператор позволяет присвоить метку любому оператору программы.

```

(mot "label" :at "name" "label name" :at "statement" "statement")

```

Пустой оператор не выполняет никаких действий.

```

(cot "empty stmt")

```

Операторы инкремента и декремента являются операторами, а не выражениями, что отличает Go от многих других языков.

```
(mot "1++ stmt" :at 1 "expression")
(mot "1-- stmt" :at 1 "expression")
```

Операторы присваивания включают как простое присваивание, так и составные с арифметическими и битовыми операциями.

```
(typedef "assignment stmt"
  (uniont "1=2" "1+=2" "1-=2" "1*=2" "1/=2" "1%=2"
    "1<<=2" "1>>=2" "1^=2" "1|=2" "1&=2" "1&^=2"))

(mot "1=2" :at 1 (listt "expression") :at 2 (listt "expression"))
(mot "1+=2" :at 1 (listt "expression") :at 2 (listt "expression"))
(mot "1-=2" :at 1 (listt "expression") :at 2 (listt "expression"))
(mot "1*=2" :at 1 (listt "expression") :at 2 (listt "expression"))
(mot "1/=2" :at 1 (listt "expression") :at 2 (listt "expression"))
(mot "1%=2" :at 1 (listt "expression") :at 2 (listt "expression"))
(mot "1<<=2" :at 1 (listt "expression") :at 2 (listt "expression"))
(mot "1>>=2" :at 1 (listt "expression") :at 2 (listt "expression"))
(mot "1^=2" :at 1 (listt "expression") :at 2 (listt "expression"))
(mot "1|=2" :at 1 (listt "expression") :at 2 (listt "expression"))
(mot "1&=2" :at 1 (listt "expression") :at 2 (listt "expression"))
(mot "1&^=2" :at 1 (listt "expression") :at 2 (listt "expression"))
```

Условный оператор может содержать необязательную инициализирующую часть.

```
(mot "if stmt"
  :at "init" "statement"
  :at "condition" "expression"
  :at "then" "block"
  :at "else" (uniont "if stmt" "block"))
```

Оператор выбора существует в двух формах.

```
(typedef "switch stmt" (uniont "expr switch stmt" "type switch stmt"))

(cot "default")

(cot "fallthrough")
```

Первая, переключатель по выражению.

```
(mot "expr switch stmt"
  :at "init"      "statement"
  :at "controlled" "expression"
  :at "cases"     (listt "expr case clause"))

(mot "expr case clause"
  :at "cases"     (uniont (listt "expression") "default")
  :at "statements" (listt "statement"))
```

Вторая, переключатель по типу.

```
(mot "type switch stmt"
  :at "init"      "statement"
  :at "name"      "variable name"
  :at "value"     "expression"
  :at "cases"     (listt "type case clause"))

(mot "type case clause"
  :at "cases"     (uniont (listt "type") "default")
  :at "statements" (listt "statement"))
```

Оператор цикла может быть представлен в нескольких формах.

```
(typedef "for stmt" (uniont "for condition" "for clause"
  "for range indexable" "for range map"
  "for range channel" "for range int"))
```

Классический цикл с условием. В языке программирования C и ему подобных аналогичная конструкция обозначается ключевым словом `while`.

```
(mot "for condition"  
  :at "condition" "expression"  
  :at "body"      "block")
```

Цикл с инициализацией, условием и пост-действием.

```
(mot "for clause"  
  :at "init"      "statement"  
  :at "condition" "expression"  
  :at "post"      "statement"  
  :at "body"      "block")
```

Цикл `range` для массивов, срезов или строк.

```
(mot "for range indexable"  
  :at "index"      "variable name"  
  :at "value"      "variable name"  
  :at "operation" (enumt "assign" "decl")  
  :at "indexable" "expression"  
  :at "body"      "block")
```

Цикл `range` для карт.

```
(mot "for range map"  
  :at "key"        "variable name"  
  :at "value"      "variable name"  
  :at "operation" (enumt "assign" "decl")  
  :at "map"        "expression"  
  :at "body"      "block")
```

Цикл `range` для каналов.

```
(mot "for range channel"
  :at "value"      "variable name"
  :at "operation" (enumt "assign" "decl")
  :at "channel"    "expression"
  :at "body"      "block")
```

Цикл `range` для перечисления отрезка натурального ряда до заданного числа.

```
(mot "for range int"
  :at "value"      "variable name"
  :at "operation" (enumt "assign" "decl")
  :at "max"        "expression"
  :at "body"      "block")
```

Операторы передачи управления включают `return`, `break`, `continue` и `goto`.

```
(mot "return stmt"  :at 1 (listt "expression"))
(mot "break stmt"   :at 1 "label name")
(mot "continue stmt":at 1 "label name")
(mot "goto stmt"    :at 1 "label name")
```

Оператор `defer` откладывает выполнение функции до возврата из окружающей функции. Оператор `go` запускает функцию в отдельной горутине.

```
(mot "defer stmt" :at 1 "expression")
(mot "go stmt"    :at 1 "expression")
```

Оператор отправки данных в канал.

```
(mot "l<-r" :at 1 "expression" :at 2 "expression")
```

Оператор `select` для работы с несколькими каналами.

```
(mot "select stmt" :at "cases" (listt "common clause"))
```

```
(mot "common clause"
  :at "case"      (uniont "send stmt" "receive assign" "receive
decl" "default")
  :at "statements" (listt "statement"))

(mot "receive assign" :at 1 (listt "expression") :at 2 "expression")
(mot "receive decl"   :at 1 (listt "variable name") :at 2 "expression")
```

3. Операционная семантика выражений

Операционная семантика определяет, как вычисляются выражения языка Go в процессе выполнения программы. В данном разделе на языке ABML формализуются правила вычисления для всех конструкций языка Go.

Вычисление происходит в контексте окружения — агента, который хранит информацию о переменных, константах, функциях и типах. Каждый шаг вычисления представлен декларацией аспекта (`aspect`), которая задает атрибут `opsem::rvalue` для выражений, возвращающих значения, `opsem::lvalue` для получения ячеек памяти, и `opsem` для конструкций языка, не возвращающих значений.

3.1. Окружение и агенты

Окружение представляет собой совокупность агентов, каждый из которых моделирует состояние выполнения программы. Атрибут `value` автоматически заполняется последним вычисленным этим агентом значением. Остальные 5 атрибутов хранят отношения декларируемых в программе имен к значениям констант, переменных, функций, методов и пользовательских типов соответственно.

```
(mot "env" :at "agents" (listt "agent"))

(mot "agent"
  :at "constant value" (mot :amap "constant name" "Go value"))
```

```

:at "variable cell" (mot :amap "variable name" "cell")
:at "function value" (mot :amap "function name" "function value")
:at "method value" (mot :amap "type name"
                      (mot :amap "method name" "method value"))
:at "type" (mot :amap "type name" "type")
:at "value" "Go value")

```

3.2. Базовое правило для значений

Если выражение уже является значением, оно возвращается без изменений.

```
(aspect "opsem::rvalue" :context ac :type "Go value" :instance i :do i)
```

3.3. Литералы

Здесь и далее будет описываться вычисление соответствующих конструкций онтологической модели. Повторно приводиться их описание не будет.

3.3.1. Литерал массива

Вычисление литерала массива происходит в несколько этапов. Сначала вычисляется значение по умолчанию для элементов массива. Соответствующая информация получается из типа литерала.

```

(aspect "opsem::rvalue" :context ac :type "array lit" :instance i
  :stage nil :do

  (update-push-acontext ac :stage "fill defaults")
  (clear-update-eval-acontext ac :aspect "default value by type"
    :instance (aget i "type" "elem type"))

```

Затем, на стадии `fill defaults` создается массив из ячеек памяти, содержащих значения по умолчанию.

```

(aspect "opsem::rvalue" :context ac :type "array lit" :instance i
  :stage "fill defaults" :ap i "elements" es :ap i "type" at
  :ap at "elem type" et :p (mo "pointer type" :av "type" et) ct
  :value v :do

  (match :v (null es) T
    :do (mo "array value" :av "type" at :av "elements" nil)
    :exit (update-push-acontext ac :stage "evaluating" :av "left" es
      :av "array" (let ((c nil))
        (dotimes (k (aget at "len") c) (setf c
          (cons (mo "cell" :av "value" v :av "type" ct) c))))))
    (clear-update-eval-acontext ac
      :instance (aget (car es) "value"))))

```

На стадии `evaluating` происходит замена значений в соответствующих ячейках массива. Сначала изменяет k -й элемент массива. Затем, если вычислять нечего, то создает и возвращает значение массива. Иначе рекурсивно вызывается этот же шаг для необработанных элементов.

```

(aspect "opsem::rvalue" :context ac :type "array lit" :instance i
  :stage "evaluating" :ap ac "left" left :ap ac "array" arr
  :value v :ap (car left) "index" k :do

  (let ((c arr)) (dotimes (j k (setf (car c) "value" v))
    (setf c (cdr c))))
  (match :v (null (cdr left)) T
    :do (mo "array value" :av "type" (aget i "type") :av "elements" arr)
    :exit (update-push-acontext ac :av "left" (cdr left)
      :av "array" arr)
    (clear-update-eval-acontext ac
      :instance (aget (car (cdr left)) "value"))))

```

3.3.2. Литерал среза

Вычисление литерала среза происходит аналогично вычислению массива. Сначала вычисляется значение по умолчанию.

```
(aspect "opsem::rvalue" :context ac :type "slice lit" :instance i
  :stage nil :do

  (update-push-acontext ac :stage "build value")
  (update-push-acontext ac :stage "fill defaults")
  (clear-update-eval-acontext ac :aspect "default value by type"
    :instance (aget i "type" "elem type"))))
```

На стадии `fill defaults` по максимальному индексу `m` создается нужного размера массив, заполненный ячейками со значениями по умолчанию.

```
(aspect "opsem::rvalue" :context ac :type "slice lit" :instance i
  :stage "fill defaults" :value v :ap i "elements" es :p -1 m
  :p (mo "pointer type" :av "type" (mo "pointer type"
    :av "type" (aget i "type" "elem type"))) ct :do

  (dolist (e es) (let ((k (aget e "index"))) (if (> k m) (setf m k))))
  (match :v (null es) T
    :do nil
    :exit (update-push-acontext ac :stage "evaluating" :av "left" es
      :av "array" (let ((a nil)) (dotimes (k (1+ m)) a)
        (setf a (cons (mo "cell" :av "value" v :av "type" ct) a)))))
    (clear-update-eval-acontext ac
      :instance (aget (car es) "value"))))
```

На стадии `evaluating`, после вычисления всех явно заданных элементов, возвращает их список в правильном порядке.

```
(aspect "opsem::rvalue" :context ac :type "slice lit" :instance i
```

```

:stage "evaluating" :ap ac "left" left :ap ac "array" arr
:value v :p (aget (car left) "index") k :do

(let ((c arr))
  (dotimes (j k (setf (car c) "value" v)) (setf c (cdr c))))
(match :v (null left) T
:do (reverse arr)
:exit (update-push-acontext ac :av "left" (cdr left) :av "array" arr)
      (clear-update-eval-acontext ac
        :instance (aget (car (cdr left)) "value"))))

```

На последней стадии `build value` создается значение среза, ссылающееся на созданный массив.

```

(aspect "opsem::rvalue" :context ac :type "slice lit" :instance i
:stage "build value" :value arr :ap i "type" stp
:ap stp "elem type" etp :p (length arr) n :do

(mo "slice value"
:av "type" (aget i "type")
:av "array" (mo "array value" :av "type" (mo "array type"
:av "elem type" etp :av "len" n) :av "elements" arr)
:av "offset" 0
:av "length" n
:av "capacity" n))

```

3.3.3. Литерал структуры

Если структура имеет хотя бы одно поле, то вычисляет значение по умолчанию для первого поля структуры и отправляет на следующую стадию.

```

(aspect "opsem::rvalue" :context ac :type "struct lit" :instance i
:stage nil :p (aget i "type" "fields") tfs

```

```

:p (attributes tfs) tns :do

(update-push-acontext ac :stage "build value")
(update-push-acontext ac :stage "eval field values")
(match :v (null ns) T :do nil
:exit (update-push-acontext ac
      :stage "filling defaults"
      :av "type field names" tns
      :av "struct value" (mo :amap "field name" "cell"))
      (clear-update-eval-acontext ac
      :aspect "default value by type"
      :instance (aget tfs (car tns))))))

```

На стадии `filling defaults` все поля получают значения по умолчанию в соответствии с собственным типом.

```

(aspect "opsem::rvalue" :context ac :type "struct lit" :instance i
:stage "filling defaults" :ap ac "type field names" tns
:ap ac "struct value" sv :value dv
:p (mo "pointer type" :av "type" (mo "pointer type" :av "type"
  (aget i "type" "fields" (car tns)))) ct :do

(aset sv (car tns) (mo "cell" :av "value" dv :av "type" ct))
(match :v (null tns) T
:do sv
:exit (update-push-acontext ac :av "type field names" (cdr tns)
      :av "struct value" sv)
      (clear-update-eval-acontext ac :aspect "default value by type"
      :instance (aget (aget i "type" "fields") (car (cdr tns))))))

```

На стадии `eval field values` все явно перечисленные поля получают заданные значения.

```

(aspect "opsem::rvalue" :context ac :type "struct lit" :instance i
  :stage "eval field values" :value sv
  :ap i "fields" fs :p (attributes fs) ns :do

  (match :v (null ns) T
    :do sv
    :exit (update-push-acontext ac :stage "evaluating" :av "names" ns
      :av "struct value" sv)
      (clear-update-eval-acontext ac :instance (aget fs (car ns))))))

```

На стадии `evaluating` вычисляется очередное явно заданное значение поля структуры.

```

(aspect "opsem::rvalue" :context ac :type "struct lit" :instance i
  :stage "evaluating" :ap ac "names" ns :ap ac "struct value" sv
  :ap i "fields" fs :value v :do

  (aset sv (car ns) "value" v)
  (match :v (null (cdr ns)) T
    :do sv
    :exit (update-push-acontext ac :av "names" (cdr ns)
      :av "struct value" sv)
      (clear-update-eval-acontext ac
        :instance (aget fs (car (cdr ns))))))

```

На стадии `build value` создается и возвращается значение структуры.

```

(aspect "opsem::rvalue" :context ac :type "struct lit" :instance i
  :stage "build value" :value sv :do

  (mo "struct value" :av "type" (aget i "type") :av "fields" sv))

```

3.3.4. Литерал карты

Вычисление литерала карты начинается с последовательного вычисления пар ключ-значение.

```
(aspect "opsem::rvalue" :context ac :type "map lit" :instance i
  :stage nil :ap i "elements" es :do

  (update-push-acontext ac :stage "build value")
  (match :v (null es) T
  :do nil
  :exit (update-push-acontext ac
    :stage "evaluating"
    :av "entries" (mo :amap "Go value" "cell")
    :ap "elements" (cdr es))
    (clear-update-eval-acontext ac :instance (car es)
    :av "elem type" (aget i "type" "elem type")))))
```

На стадии `evaluating` добавляются вычисленные записи.

```
(aspect "opsem::rvalue" :context ac :type "map lit" :instance i
  :stage "evaluating" :ap ac "entries" ent
  :ap ac "elements" es :value ke :do

  (aset ent (aget ke "key") (aget ke "value"))
  (match :v (null es) T :do ent
  :exit (update-push-acontext ac :av "entries" ent
    :av "elements" (cdr es))
    (update-push-acontext ac :instance (car es)))
    (clear-update-eval-acontext ac :instance (aget (car es) "key")
    :av "elem type" (aget i "type" "elem type")))))
```

На стадии `build value` создается и возвращается значение карты.

```
(aspect "opsem::rvalue" :context ac :type "map lit" :instance i
  :stage "build value" :value ent :do
```

```
(mo "map value" :av "type" (aget i "type") :av "entries" ent))
```

Теперь разберем вычисление записей карты. Оно начинается с вычисления значения.

```
(aspect "opsem::rvalue" :context ac :type "keyed elem" :instance i
  :stage nil :value k :do

  (update-push-acontext ac :stage "eval value" :av "key" k)
  (clear-update-eval-acontext ac :instance (aget i "value"))))
```

Затем, на стадии `eval value` создается и возвращается запись карты.

```
(aspect "opsem::rvalue" :context ac :type "keyed elem" :instance i
  :stage "eval value" :ap ac "key" k :ap ac "elem type" et :value v :do

  (mo "keyed value" :av "key" k :av "value"
    (mo "cell" :av "value" v
      :av "type" (mo "pointer type" :av "type" et))))))
```

3.3.5. Литерал функции

Функциональные литералы создают значения функций, сохраняя текущее состояние замыкания — ячейки памяти всех переменных, видимых в момент создания.

```
(aspect "opsem::rvalue" :context ac :type "function lit" :instance i
  :agent a :p (aget i "body" "all variables") ns :p nil c :do

  (dolist (item ns) (setf c (aset c (aget a "variable cell" item)))))
  (mo "function value" :av "type" (aget i "type")
    :av "signature" (aget i "signature")
    :av "body" (aget i "body")
    :av "closure" c))
```

3.3.6. Литерал метода

Аналогично создается значение метода.

```
(aspect "opsem::rvalue" :context ac :type "method lit" :instance i
  :agent a :p (aget i "body" "all variables") ns :p nil c :do

  (dolist (item ns) (setf c (aset c (aget a "variable cell" item)))))

  (mo "method value" :av "type" (aget i "type")
    :av "signature" (aget i "signature")
    :av "body" (aget i "body")
    :av "closure" c))
```

3.4. Выражения

3.4.1. Ссылка на переменную

Для ссылки на переменную определены два варианта вычисления: для получения левого значения (lvalue) — ячейки памяти, и для получения правого значения (rvalue) — содержимого ячейки.

```
(aspect "opsem::lvalue" :context ac :type "variable ref" :instance i
  :agent a :do (aget a "variable cell" i))

(aspect "opsem::rvalue" :context ac :type "variable ref" :instance i
  :agent a :do (aget (aget a "variable cell" i) "value"))
```

3.4.2. Выражение в скобках

Выражение в скобках вычисляет свое содержимое.

```
(aspect "opsem::rvalue" :context ac :type "(1)" :instance i :do
  (clear—update—eval—acontext ac :instance (aget i 1)))
```

3.4.3. Приведение типов

Вычисление выражения приведения типов начинается с вычисления значения, которое необходимо привести к другому типу.

```
(aspect "opsem::rvalue" :context ac :type "conversion" :instance i
  :stage nil :do

  (update—push—acontext ac :stage "type from")
  (clear—update—eval—acontext ac :instance (aget i "value")))
```

На стадии `type from` происходит определение типа вычисленного значения.

```
(aspect "opsem::rvalue" :context ac :type "conversion" :instance i
  :stage "type from" :value tv :ap tv "type" ttv :do

  (update—push—acontext ac :stage "build value")
  (clear—update—eval—acontext ac :av "typed value" tv :stage (nmatch
:v (is—instance ttv "int type")    T :exit "int type"
:v (is—instance ttv "float type")  T :exit "float type"
:v (is—instance ttv "complex type") T :exit "complex type"
:v (is—instance ttv "string type") T :exit "string type"
:v (is—instance ttv "slice type")  T :exit "[ ] byte type")))
```

На стадии `int type` рассматривается приведение целого числа к заданному типу. При чем целое число можно привести только к численным типам: знаковые, беззнаковые или вещественные.

```
(aspect "opsem::rvalue" :context ac :type "conversion" :instance i
  :stage "int type" :ap ac "typed value" tv :ap tv "value" v
  :ap i "type" tp :ap tp "bit size" bs
  :p (logand v (1— (ash 1 bs))) x :do

  (nmatch
:v (is—instance tp "signed int type")  T
:exit (if (> x (ash 1 (1— bs))) (— x (ash 1 bs)) x)
```

```
:v (is—instance tp "unsigned int type") T :exit x
:v (is—instance tp "float type") T :exit (float v 0.0d0)))
```

На стадии `float type` описывается приведение вещественного числа к заданному типу. Причем вещественное число можно привести только к таким же численным типам, что и в целочисленном случае.

```
(aspect "opsem::rvalue" :context ac :type "conversion" :instance i
  :stage "float type" :ap ac "typed value" tv :ap tv "value" v
  :ap i "type" tp :ap tp "bit size" bs
  :p (logand v (1— (ash 1 bs))) x :do

  (nmatch
    :v (is—instance tp "signed int type") T
    :exit (let ((p (truncate x 1))) (if (> p (ash 1 (1— bs))) (— p (ash
      1 bs)) p))
    :v (is—instance tp "unsigned int type") T :exit (truncate x 1)
    :v (is—instance tp "float type") T :exit v))
```

На стадии `complex type` описывается приведение комплексных чисел. Однако, допускается приведение только между комплексными типами данных. В данной реализации это не представляет разницы, так как все дробные числа хранятся в формате double-float языка lisp.

```
(aspect "opsem::rvalue" :context ac :type "conversion" :instance i
  :stage "complex type" :value v :do v)
```

На стадии `string type` нетривиальным случаем является приведение строки к типу среза `byte`. Напомню, что этот тип является аналогом `uint8`.

```
(aspect "opsem::rvalue" :context ac :type "conversion" :instance i
  :stage "string type" :ap ac "typed value" tv :ap tv "value" v
  :ap i "type" tp :do
```

```
(nmatch
:~ (is-instance tp "string type") T :exit v
:do (mapcar #'char-code (coerce v 'list))))
```

На стадии `[]byte type` в нетривиальном случае сначала строится список `lst` значений полученного среза. Потом этот список преобразуется в строку, которая и возвращается.

```
(aspect "opsem::rvalue" :context ac :type "conversion" :instance i
:stage "[]byte type" :ap ac "typed value" tv :ap tv "value" sv
:ap i "type" tp :ap sv "offset" ofs :do

(nmatch :v (is-instance tp "string type") nil :exit sv
:p nil lst :p (aget sv "array" "elements") cs :do
(dotimes (k ofs) (setf cs (cdr cs)))
(dotimes (k (aget sv "length") (reverse lst))
(setf lst (cons (aget (car cs) "value") lst)) (setf cs (cdr cs)))
(coerce (mapcar #'code-char lst) 'string)))
```

На стадии `build value` возвращаемое значение оборачивается в объект `typed primitive` для поддержания модели типизированных значений. Причем в случае приведения к типу среза это не требуется.

```
(aspect "opsem::rvalue" :context ac :type "conversion" :instance i
:stage "build value" :value v :ap i "type" tp
:v (is-instance tp "slice type") nil :do

(mo "typed primitive" :av "type" tp :av "value" v))
```

3.4.4. Метод-выражение

Метод-выражение преобразует метод в функцию, где получатель становится первым параметром.

```
(aspect "opsem::rvalue" :context ac :type "method expr" :instance i
```

```

:p (aget a "method value" (aget i "receiver type") (aget i "name")) mv
:agent a :ap mv "type" mt :ap mv "signature" ms :do

(mo "function value"
:av "type"      (mo "function type"
  :av "param types"      (cons (aget mt "receiver type")
                                (aget mt "param types")))
  :av "variadic type" (aget mt "variadic type")
  :av "result type"   (aget mt "result types"))
:av "signature" (mo "function signature"
  :av "parameters"      (cons (aget ms "receiver")
                                (aget ms "parameters")))
  :av "variadic param" (aget ms "variadic param")
  :av "result"         (aget ms "result"))
:av "body"          (aget mv "body")
:av "closure"       (aget mv "closure"))))

```

3.4.5. Выбор поля или метода

Выбор поля или метода обрабатывает два случая: когда слева от точки стоит имя типа (интерфейса) — возвращается метод; когда слева стоит структура или указатель на структуру — возвращается поле.

Если слева от точки стоит имя пользовательского типа (в том числе интерфейса), то вычислять ничего не нужно

```

(aspect "opsem::rvalue" :context ac :type "selector expr" :instance i
  :stage nil :ap i "receiver" r :do

  (update-push-acontext ac :stage "return value")
  (match :v (and (is-instance r "identifier") (aget a "type" r)) T
  :do (aget a "method value" r (aget i "name"))
  :exit (clear-update-eval-acontext ac :instance (aget i "receiver"))))

```

Иначе слева стоит структура, или указатель на нее, который сначала нужно разыменовать.

```
(aspect "opsem::rvalue" :context ac :type "selector expr" :instance i
  :stage "return value" :value r :ap r "type" tr :ap i "name" n :do

  (nmatch
    :v tr "struct type" :exit (aget r "fields" n "value")
    :v tr "pointer type" :exit (aget r "target" "value" "fields" n
      "value"))))
```

3.4.6. Индексирование

Сначала вычисляем хранилище, к которому применимо индексирование: массив, срез, карта или строка.

```
(aspect "opsem::rvalue" :context ac :type "index expr" :instance i
  :stage nil :do

  (update-push-acontext ac :stage "eval index")
  (clear-update-eval-acontext ac :instance (aget i "indexable"))))
```

На стадии `eval index` вычисляем индекс, по которому хотим обратиться к хранилищу.

```
(aspect "opsem::rvalue" :context ac :type "index expr" :instance i
  :stage "eval index" :value indl :do

  (update-push-acontext ac :stage "return value" :av "indexable" indl)
  (clear-update-eval-acontext ac :instance (aget i "index"))))
```

На стадии `return value` возвращается искомое значение, доступ к которому получается в зависимости от типа хранилища.

```
(aspect "opsem::rvalue" :context ac :type "index expr"
  :stage "return value" :value index :ap ac "indexable" indl
  :ap indl "type" it :do
```

```

(nmatch
:~ it "array type"
:~exit (aget (nth index (aget indl "elements"))) "value")
:~ it "slice type"
:~exit (aget (nth (+ (aget indl "offset") index)
                  (aget indl "array" "elements"))) "value")
:~ it "map type"
:~exit (aget (aget indl "entries" index) "value")
:~ it "string type"
:~exit (char (aget indl "value") index)))

```

Вычисления этого выражения в lvalue позиции происходит аналогичным образом, за исключением того, что к строке индексирование применяться не может.

```

(aspect "opsem::lvalue" :context ac :type "indexable expr" :instance i
:stage nil :do

(update-push-acontext ac :stage "eval index")
(clear-update-eval-acontext ac :instance (aget i "indexable")
:aspect "opsem::rvalue"))
(aspect "opsem::lvalue" :context ac :type "index expr" :instance i
:stage "eval index" :value indl :do

(update-push-acontext ac :stage "return value" :av "indexable" indl)
(clear-update-eval-acontext ac :instance (aget i "index")
:aspect "opsem::rvalue"))
(aspect "opsem::lvalue" :context ac :type "index expr"
:stage "return value" :value index :ap ac "indexable" indl
:ap indl "type" it :do

(nmatch

```

```

:~v it "array type" :exit (nth index (~aget indl "elements"))
:~v it "slice type"
:exit (nth (+ offset index) (~aget indl "array" "elements"))
:~v it "map type" :exit (~aget indl "entries" index)))

```

3.4.7. Выражение среза

Выражение среза создает новый срез из существующего массива, среза или строки.

Сначала вычисляем все атрибуты: `sequence`, `low`, `high`, `max`.

```

(aspect "opsem::rvalue" :context ac :type "slice expr" :instance i
  :stage nil :do

  (update-push-acontext ac :stage "eval low")
  (clear-update-eval-acontext ac :instance (~aget i "sequence")))

(aspect "opsem::rvalue" :context ac :type "slice expr" :instance i
  :stage "eval low" :value s :do

  (update-push-acontext ac :stage "eval high" :av "sequence" s)
  (clear-update-eval-acontext ac :instance (~aget i "low")))

(aspect "opsem::rvalue" :context ac :type "slice expr" :instance i
  :stage "eval high" :ap ac "sequence" s :value low :do

  (update-push-acontext ac :stage "eval max" :av "sequence" s
    :av "low" (~if low low 0))
  (clear-update-eval-acontext ac :instance (~aget i "high")))

(aspect "opsem::rvalue" :context ac :type "slice expr" :instance i
  :stage "eval max" :ap ac "sequence" s :ap ac "low" low :value high :do

```

```
(update-push-acontext ac :stage "build value" :av "sequence" s
      :av "low" low :av "high" high)
(clear-update-eval-acontext ac :instance (aget i "max")))
```

На стадии `build value` определяем тип хранилища.

```
(aspect "opsem::rvalue" :context ac :type "slice expr"
      :stage "build value" :ap ac "sequence" s :ap ac "low" low
      :ap ac "high" high :value cap :ap s "type" st :do

(clear-update-eval-acontext ac :stage (otype st) :av "sequence" s
      :av "low" low :av "high" high :av "cap" cap))
```

На стадии `array type` описывается взятие среза по массиву.

```
(aspect "opsem::rvalue" :context ac :type "slice expr"
      :stage "array type" :ap ac "sequence" s :ap ac "low" low
      :ap ac "high" :ap ac "cap" cap :ap s "type" st
      :p (length (aget s "elements")) len :do

(if (not high) (aset high len))
(if (not cap) (aset cap (- len low)))
(mo "slice value"
  :av "type" (mo "slice type" :av "elem type" (aget st "elem type"))
  :av "array" s
  :av "offset" low
  :av "length" high - low
  :av "capacity" cap)
```

На стадии `pointer type` описывается взятие среза по указателю на массив.

```
(aspect "opsem::rvalue" :context ac :type "slice expr"
      :stage "pointer type" :ap ac "sequence" s :ap ac "low" low
      :ap ac "high" high :ap ac "cap" cap :do
```

```
(clear—update—eval—acontext ac :stage "array type"
      :av "sequence" (aget s "value")))
```

На стадии `array type` описывается взятие среза по срезу.

```
(aspect "opsem::rvalue" :context ac :type "slice expr"
  :stage "slice type" :ap ac "sequence" s :ap ac "low" low
  :ap ac "high" high :ap ac "cap" cap
  :p (length (aget s "array" "type" "len")) len :do

  (if (not high) (aset high len))
  (if (not cap) (aset cap (— len low)))
  (mo "slice value"
    :av "type" (aget s "type")
    :av "array" (aget s "array")
    :av "offset" (+ (aget s "offset") low)
    :av "length" (aget s "length")
    :av "capacity" (aget s "capacity")))
```

На стадии `array type` описывается взятие среза по строке.

```
(aspect "opsem::rvalue" :context ac :type "slice expr"
  :stage "string type" :p (aget ac "sequence" "value") s
  :ap ac "low" low :ap ac "high" high :do

  (if (not high) (aset high (length s)))
  (subseq s low high))
```

3.4.8. Вызов функции

Вычисление этого выражения начинается с вычисления самой функции. Например, это может быть имя или литерал функции.

```
(aspect "opsem::rvalue" :context ac :type "function call" :instance i
  :stage nil :do

  (update-push-acontext ac :stage "eval arguments")
  (clear-update-eval-acontext ac :instance (aget i "function"))))
```

Затем вычисляются аргументы функции.

```
(aspect "opsem::rvalue" :context ac :type "function call" :instance i
  :stage "eval arguments" :value f :ap i "arguments" as :do

  (update-push-acontext ac :stage "copy parameters" :av "function" f)
  (match :v (null as) T :do nil
  :exit (update-push-acontext ac :stage "evaluating" :av "arguments" nil
    :av "left" (cdr as))
    (clear-update-eval-acontext ac :instance (car as))))
```

Непосредственно вычисление параметров функции.

```
(aspect "opsem::rvalue" :context ac :type "function call"
  :stage "evaluating" :ap ac "arguments" as :ap ac "left" left
  :value v :p (cons v as) nas :do

  (match :v (null left) T :do (reverse nas)
  :exit (update-push-acontext ac :av "arguments" (cdr as)
    :av "left" (cdr left))
    (clear-update-eval-acontext ac :instance (car as))))
```

Затем создаются копии значений всех параметров, с которыми и будет происходить работа в теле функции.

```
(aspect "opsem::rvalue" :context ac :type "function call"
  :stage "copy parameters" :ap ac "function" f :ap f "signature" sgn
```

```

:ap sgn "result" r :value as :agent a :p nil dcs :do

(let ((args as)) (dolist (p (aget sgn "parameters"))
  (setf dcs (cons (aget a "variable cell" (aget p "name")) dcs))

  (aset a "variable cell" (aget p "name") (mo "cell" :av "type"
    (aget p "type") :av "value" (car args))))
  (setf args (cdr args))))

(update-push-acontext ac :stage "copy closure" :av "function" f)
(match :v (null r) nil :v (is-instance (car r) "param decl") T
:do (update-push-acontext ac :stage "copying result" :av "left" r
  :av "done" dcs)

  (clear-update-eval-acontext ac :instance (aget (car r) "type")
    :aspect "default value by type")

:exit dcs))

```

Непосредственно вычисление копий значений параметров функции.

```

(aspect "opsem::rvalue" :context ac :type "function call"
:stage "copying result" :ap ac "left" l :ap ac "done" done :value v
:agent a :ap (car l) "name" n
:p (cons (aget a "variable cell" n) done) nd :do

(aset a "variable cell" n (mo "cell" :av "type" (aget (car l)
  "type") :av "value" v))
(match :v (null l) T :do nd
:exit

(update-push-acontext ac :av "left" (cdr "left") :av "done" nd)
(clear-update-eval-acontext ac :instance (aget (car l) "type")
  :aspect "default value by type"))))

```

Затем необходимо так же скопировать контекст функции, что бы иметь возможность об-

ращаться к переменным, объявленным вне функции, но в области ее видимости.

```
(aspect "opsem::rvalue" :context ac :type "function call"
  :stage "copy closure" :ap ac "function" f :value dcs
  :ap f "closure" cl :agent a :do

  (dolist (n (attributes cl)) (setf dcs (cons (aget cl n) dcs))
    (setf a "variable cell" n
      (mo "cell" :av "type" (aget cl n "type")
        :av "value" (aget cl n "value")))))
  (update-push-acontext ac :stage "variable back" :av "function" f
    :av "cells" (reverse dcs))
  (clear-update-eval-acontext ac :instance (aget f "body")))
```

Наконец, после вызова функции, происходит возврат значений всех возможно затененных переменных к прежним значениям.

```
(aspect "opsem::rvalue" :context ac :type "function call"
  :stage "variable back" :ap ac "function" f :ap ac "cells" cs
  :ap f "signature" sgn :ap sgn "result" r :ap f "closure" cl
  :value v :do

  (dolist (p (aget sgn "parameters")) (aset a "variable cell"
    (aget p "name") (car cs)) (setf cs (cdr cs)))
  (match :v (null r) nil :v (is-instance (car r) "param decl") T
    :do (dolist (p r) (aset a "variable cell" (aget p "name") (car cs))
      (setf cs (cdr cs))))
  )
  (dolist (n (attributes cl)) (setf a "variable cell" n (car cs))
    (setf cs (cdr cs)))
  (car v))
```

Вызов метода очень похож на вызов функции. Поэтому будем сводить этот случай к предыдущему.

Сначала вычисляем получателя метода.

```
(aspect "opsem::rvalue" :context ac :type "method call" :instance i
  :stage nil :do

  (update—push—acontext ac :stage "to function")
  (clear—update—eval—acontext ac :instance (aget i "receiver"))))
```

Затем, используем ранее определенное выражение преобразования метода в функцию.

```
(aspect "opsem::rvalue" :context ac :type "method call" :instance i
  :stage "to function" :value rec :do

  (update—push—acontext ac :stage "eval call")
  (clear—update—eval—acontext ac :instance (mo "method expr"
    :av "receiver type" (aget rec "type") :av "name" (aget i
      "method")))))
```

Наконец, имея функцию, выполняем вызов функции, что описано в предыдущей секции.

```
(aspect "opsem::rvalue" :context ac :type "method call" :instance i
  :stage "eval call" :value f :do

  (clear—update—eval—acontext ac :instance (mo "function call"
    :av "function" f :av "arguments" (aget i "arguments")))))
```

3.4.10. Унарные выражения

Все унарные выражения сначала вычисляют свой операнд, затем применяют операцию.

```
(aspect "opsem::rvalue" :context ac :type "unary expression" :instance i
  :stage nil :do
```

```
(update-push-acontext ac :stage "apply")
(clear-update-eval-acontext ac :instance (aget i 1)))
```

Вспомогательная функция для приведения целых чисел к нужному типу с учетом переполнения (wrap-around для беззнаковых, знаковое представление для знаковых).

```
(defun convert-int-to-type (tp raw)
  (mo "typed primitive" :av "type" tp :av "value"
    (if (not (is-instance tp "int type")) raw
        (let* ((bs (aget tp "bit size")) (v (mod raw (ash 1 bs))))
          (if (or (is-instance tp "unsigned int type")
                  (< v (ash 1 (1- bs))))
              v
              (- v (ash 1 (1- bs))))))))
```

Унарный плюс.

```
(aspect "opsem::rvalue" :context ac :type "+1" :stage "apply"
  :value v :do v)
```

Унарный минус.

```
(aspect "opsem::rvalue" :context ac :type "-1" :stage "apply"
  :value pr :ap pr "value" v :ap pr "type" tp
  :do (convert-int-to-type tp (- v)))
```

Побитовое отрицание.

```
(aspect "opsem::rvalue" :context ac :type "^1" :stage "apply"
  :value pr :ap pr "value" v :ap pr "type" tp
  :do (convert-int-to-type tp (lognot v))
)
```

Логическое отрицание.

```
(aspect "opsem::rvalue" :context ac :type "!1" :stage "apply"
  :value v :do
    (mo "typed primitive" :av "type" (aget v "type")
      :av "value" (not (aget v "value")))))
```

Разыменование указателя (может быть как rvalue, так и lvalue). Применимо только к ячейкам памяти `cell`

```
(aspect "opsem::rvalue" :context ac :type "*1" :stage "apply"
  :value tv :ap tv "value" v :do
    (mo "typed primitive" :av "type" (aget tv "type")
      :av "value" (aget v "value"))))

(aspect "opsem::lvalue" :context ac :type "*1" :instance i :stage nil
:do (update—push—acontext ac :stage "apply")
    (clear—update—eval—acontext ac :instance (aget i 1)))

(aspect "opsem::lvalue" :context ac :type "*1" :stage "apply"
  :value v :do (aget v "value"))
```

Взятие адреса.

```
(aspect "opsem::rvalue" :context ac :type "&1" :instance i
  :stage "apply" :value tv :ap tv "type" tp :ap tv "value" v :do

    (mo "typed primitive" :av "type" (co "pointer type" :av "type" tp)
      :av "value" (mo "cell" :av "value" v :av "type" tp)))
```

3.4.11. Бинарные выражения

Бинарные выражения вычисляют левый операнд, затем правый (кроме логических операций с коротким замыканием). Допускаются только бинарные выражения с операндами

одного типа.

```
(aspect "opsem::rvalue" :context ac :type "binary expression"
  :instance i
  :stage nil :do

  (update-push-acontext ac :stage "apply 1")
  (clear-update-eval-acontext ac :instance (aget i 1)))
```

Логические И/ИЛИ не вычисляют второй операнд, если его значение уже не может повлиять на результат выражения.

```
(aspect "opsem::rvalue" :context ac :type "1&&2" :instance i
  :stage "apply 1" :value v :do
  (match :v (aget v "value") nil
  :do (mo "typed primitive" :av "type" (co "bool type") :av "value" nil)
  :exit (update-push-acontext ac :stage "apply 2")
    (clear-update-eval-acontext ac :instance (aget i 2))))

(aspect "opsem::rvalue" :context ac :type "1&&2"
  :stage "apply 2" :value v :do
  (mo "typed primitive" :av "type" (co "bool type")
    :av "value" (aget v "value"))))

(aspect "opsem::rvalue" :context ac :type "1||2" :instance i
  :stage "apply 1" :value v :do

  (match :v (aget v "value") T
  :do (mo "typed primitive" :av "type" (co "bool type") :av "value" T)
  :exit (update-push-acontext ac :stage "apply 2")
    (clear-update-eval-acontext ac :instance (aget i 2))))

(aspect "opsem::rvalue" :context ac :type "1||2"
```

```

:stage "apply 2" :value v :do
  (mo "typed primitive" :av "type" (co "bool type")
    :av "value" (aget v "value")))

```

Арифметическое сложение чисел.

```

(aspect "opsem::rvalue" :context ac :type "1+2" :instance i
  :stage "apply 1" :value v :do
    (update-push-acontext ac :stage "apply 2" :av 1 v)
    (clear-update-eval-acontext ac :instance (aget i 2)))

(aspect "opsem::rvalue" :context ac :type "1+2" :stage "apply 2"
  :ap ac 1 f :ap f "type" tp :ap f "value" fv :value s
  :ap s "value" sv :do
    (convert-int-to-type tp (+ fv sv)))

```

Арифметическое вычитание двух чисел.

```

(aspect "opsem::rvalue" :context ac :type "1-2" :instance i
  :stage "apply 1" :value v :do
    (update-push-acontext ac :stage "apply 2" :av 1 v)
    (clear-update-eval-acontext ac :instance (aget i 2)))

(aspect "opsem::rvalue" :context ac :type "1-2" :stage "apply 2"
  :ap ac 1 f :ap f "type" tp :ap f "value" fv :value s
  :ap s "value" sv :do
    (convert-int-to-type tp (- fv sv)))

```

Арифметическое умножение двух чисел.

```

(aspect "opsem::rvalue" :context ac :type "1*2" :instance i
  :stage "apply 1" :value v :do
    (update-push-acontext ac :stage "apply 2" :av 1 v)
    (clear-update-eval-acontext ac :instance (aget i 2)))

```

```
(aspect "opsem::rvalue" :context ac :type "1*2" :stage "apply 2"
  :ap ac 1 f :ap f "type" tp :ap f "value" fv :value s
  :ap s "value" sv :do
  (convert-int-to-type tp (* fv sv)))
```

Арифметическое деление двух чисел. При условии корректно написанной программы предполагается невозможность второго операнда быть нулем.

```
(aspect "opsem::rvalue" :context ac :type "1/2" :instance i
  :stage "apply 1" :value v :do
  (update-push-acontext ac :stage "apply 2" :av 1 v)
  (clear-update-eval-acontext ac :instance (aget i 2)))

(aspect "opsem::rvalue" :context ac :type "1/2" :stage "apply 2"
  :ap ac 1 f :ap f "type" tp :ap f "value" fv :value s
  :ap s "value" sv :do
  (convert-int-to-type tp (/ fv sv)))
```

Взятие остатка по модулю.

```
(aspect "opsem::rvalue" :context ac :type "1%2" :instance i
  :stage "apply 1" :value v :do
  (update-push-acontext ac :stage "apply 2" :av 1 v)
  (clear-update-eval-acontext ac :instance (aget i 2)))

(aspect "opsem::rvalue" :context ac :type "1%2" :stage "apply 2"
  :ap ac 1 f :ap f "type" tp :ap f "value" fv :value s
  :ap s "value" sv :do
  (convert-int-to-type tp (rem fv sv)))
```

Побитовый сдвиг влево.

```
(aspect "opsem::rvalue" :context ac :type "1<<2" :instance i
```

```

:stage "apply 1" :value v :do
(update-push-acontext ac :stage "apply 2" :av 1 v)
(clear-update-eval-acontext ac :instance (aget i 2)))

(aspect "opsem::rvalue" :context ac :type "1<<2" :stage "apply 2"
:ap ac 1 f :ap f "type" tp :ap f "value" fv :value s
:ap s "value" sv :do
(convert-int-to-type tp (ash fv sv)))

```

Побитовый сдвиг вправо.

```

(aspect "opsem::rvalue" :context ac :type "1>>2" :instance i
:stage "apply 1" :value v :do
(update-push-acontext ac :stage "apply 2" :av 1 v)
(clear-update-eval-acontext ac :instance (aget i 2)))

(aspect "opsem::rvalue" :context ac :type "1>>2" :stage "apply 2"
:ap ac 1 f :ap f "type" tp :ap f "value" fv :value s
:ap s "value" sv :do
(convert-int-to-type tp (ash fv (- sv))))

```

Побитовое И.

```

(aspect "opsem::rvalue" :context ac :type "1&2" :instance i
:stage "apply 1" :value v :do
(update-push-acontext ac :stage "apply 2" :av 1 v)
(clear-update-eval-acontext ac :instance (aget i 2)))

(aspect "opsem::rvalue" :context ac :type "1&2" :stage "apply 2"
:ap ac 1 f :ap f "type" tp :ap f "value" fv :value s
:ap s "value" sv :do
(convert-int-to-type tp (logand fv sv)))

```

Побитовое И-НЕТ представляет собой применение операции побитового И к первому операнду и результату применения побитового НЕТ ко второму операнду.

```
(aspect "opsem::rvalue" :context ac :type "1&^2" :instance i
  :stage "apply 1" :value v :do
    (update-push-acontext ac :stage "apply 2" :av 1 v)
    (clear-update-eval-acontext ac :instance (aget i 2)))

(aspect "opsem::rvalue" :context ac :type "1&^2" :stage "apply 2"
  :ap ac 1 f :ap f "type" tp :ap f "value" fv :value s
  :ap s "value" sv :do
    (convert-int-to-type tp (logand fv (lognot sv))))
```

Побитовое ИЛИ.

```
(aspect "opsem::rvalue" :context ac :type "1|2" :instance i
  :stage "apply 1" :value v :do
    (update-push-acontext ac :stage "apply 2" :av 1 v)
    (clear-update-eval-acontext ac :instance (aget i 2)))

(aspect "opsem::rvalue" :context ac :type "1|2" :stage "apply 2"
  :ap ac 1 f :ap f "type" tp :ap f "value" fv :value s
  :ap s "value" sv :do
    (convert-int-to-type tp (logior fv sv)))
```

Исключающее ИЛИ.

```
(aspect "opsem::rvalue" :context ac :type "1^2" :instance i
  :stage "apply 1" :value v :do
    (update-push-acontext ac :stage "apply 2" :av 1 v)
    (clear-update-eval-acontext ac :instance (aget i 2)))

(aspect "opsem::rvalue" :context ac :type "1^2" :stage "apply 2"
  :ap ac 1 f :ap f "type" tp :ap f "value" fv :value s
```

```
:ap s "value" sv :do
  (convert-int-to-type tp (logxor fv sv)))
```

Далее будут описаны операции сравнения.

Начнем с логической операции равенства.

```
(aspect "opsem::rvalue" :context ac :type "1==2" :instance i
  :stage "apply 1" :value v :do
    (update-push-acontext ac :stage "apply 2" :av 1 v)
    (clear-update-eval-acontext ac :instance (aget i 2)))

(aspect "opsem::rvalue" :context ac :type "1==2" :stage "apply 2"
  :ap ac 1 f :ap f "type" tp :ap f "value" fv :value s
  :ap s "value" sv :do
    (convert-int-to-type tp (= fv sv)))
```

Логическое неравенство.

```
(aspect "opsem::rvalue" :context ac :type "1!=2" :instance i
  :stage "apply 1" :value v :do
    (update-push-acontext ac :stage "apply 2" :av 1 v)
    (clear-update-eval-acontext ac :instance (aget i 2)))

(aspect "opsem::rvalue" :context ac :type "1!=2" :stage "apply 2"
  :ap ac 1 f :ap f "type" tp :ap f "value" fv :value s
  :ap s "value" sv :do
    (convert-int-to-type tp (not (= fv sv))))
```

Логическое строго меньше.

```
(aspect "opsem::rvalue" :context ac :type "1<2" :instance i
  :stage "apply 1" :value v :do
    (update-push-acontext ac :stage "apply 2" :av 1 v)
```

```

(clear-update-eval-acontext ac :instance (aget i 2)))

(aspect "opsem::rvalue" :context ac :type "1<2" :stage "apply 2"
  :ap ac 1 f :ap f "type" tp :ap f "value" fv :value s
  :ap s "value" sv :do
  (convert-int-to-type tp (< fv sv)))

```

Логическое нестрогое меньше.

```

(aspect "opsem::rvalue" :context ac :type "1<=2" :instance i
  :stage "apply 1" :value v :do
  (update-push-acontext ac :stage "apply 2" :av 1 v)
  (clear-update-eval-acontext ac :instance (aget i 2)))

(aspect "opsem::rvalue" :context ac :type "1<=2" :stage "apply 2"
  :ap ac 1 f :ap f "type" tp :ap f "value" fv :value s
  :ap s "value" sv :do
  (convert-int-to-type tp (<= fv sv)))

```

Логическое строго больше.

```

(aspect "opsem::rvalue" :context ac :type "1>2" :instance i
  :stage "apply 1" :value v :do
  (update-push-acontext ac :stage "apply 2" :av 1 v)
  (clear-update-eval-acontext ac :instance (aget i 2)))

(aspect "opsem::rvalue" :context ac :type "1>2" :stage "apply 2"
  :ap ac 1 f :ap f "type" tp :ap f "value" fv :value s
  :ap s "value" sv :do
  (convert-int-to-type tp (> fv sv)))

```

Логическое нестрогое больше.

```

(aspect "opsem::rvalue" :context ac :type "1>=2" :instance i

```

```

:stage "apply 1" :value v :do
(update-push-acontext ac :stage "apply 2" :av 1 v)
(clear-update-eval-acontext ac :instance (aget i 2)))

(aspect "opsem::rvalue" :context ac :type "1>=2" :stage "apply 2"
:ap ac 1 f :ap f "type" tp :ap f "value" fv :value s
:ap s "value" sv :do
(convert-int-to-type tp (>= fv sv)))

```

3.4.12. Блоки

Блоки определяют области видимости переменных. При входе в блок сохраняются старые ячейки всех переменных, объявленных в этом блоке.

```

(aspect "opsem" :context ac :type "block" :instance i :stage nil
:ap i "statements" sts :ap i "decl variables" avs :p nil dv
:agent a :do

(aset i "variable cells" (dolist
(item avs (reverse dv))
(setf dv (cons (aget a "variable cell" item) dv))))
(update-push-acontext ac :stage "variable back")
(clear-update-eval-acontext ac :stage "evaluating statement"
:av "left" sts))

```

Вычисление очередного оператора блока.

```

(aspect "opsem" :context ac :type "block" :instance i
:stage "evaluating statement" :ap ac "left" left
:v (null left) nil :ap i "statements" sts :do

(update-push-acontext ac :av "left" (cdr left))
(clear-update-eval-acontext ac :instance (car left) :av "block" i))

```

Восстановление старых ячеек при выходе из блока.

```
(aspect "opsem" :context ac :type "block" :instance i
  :stage "variable back" :do
    (dolist (item (aget i "decl variables"))
      (aset a "variable cell" item (aget item "variable cells"))))
```

3.5. Декларации

3.5.1. Декларация типа

Декларация типа добавляет определение типа в окружение агента.

```
(aspect "opsem" :context ac :type "type decl" :instance i :agent a
:do (aset a "type" (aget i "name") (aget i "type")))
```

3.5.2. Декларация переменных

Декларация переменных может происходить двумя способами: с явным указанием значений или без них (используются значения по умолчанию).

```
(aspect "opsem" :context ac :type "var decl" :instance i
  :stage nil :ap ac "block" b :ap i "names" ns :ap i "values" vs
  :ap i "types" ts :do

  (match :v (null vs) T
:do
    (update-push-acontext ac :stage "declarating defaults"
      :av "names" ns :av "types" (cdr ts) :av "block" b)
    (clear-update-eval-acontext ac :aspect "default value by type"
      :instance (car ts))
:exit
    (update-push-acontext ac :stage "evaluating values" :av "names" ns
      :av "types" ts :av "values" (cdr vs) :av "block" b))
```

```
(clear—update—eval—acontext ac :instance (car vs))))
```

В случае декларации переменных без значения по умолчанию.

```
(aspect "opsem" :context ac :type "var decl" :instance i
  :stage "declarating defaults" :ap ac "names" ns :ap ac "types" ts
  :ap ac "block" b :value v :agent a :do

  (aset b "variable cells" (car ns) (aget a "variable cell" (car ns)))
  (aset a "variable cell" (car ns)
    (mo "cell" :av "value" v
      :av "type" (co "pointer type" :av "type" (car ts))))
  (match :v (null ts) nil :do
    (update—push—acontext ac :av "names" (cdr ns) :av "types" (cdr ts)
      :av "block" b)
    (clear—update—eval—acontext ac :aspect "default value by type"
      :instance (car ts))))
```

В случае декларации переменных с явно указанными значениями.

```
(aspect "opsem" :context ac :type "var decl" :instance i
  :stage "evaluating values" :ap ac "names" ns :ap ac "values" vs
  :ap ac "types" ts :ap ac "block" b :value v :agent a :do

  (aset b "variable cells" (car ns) (aget a "variable cell" (car ns)))
  (aset a "variable cell" (car ns) (mo "cell" :av "value" v
    :av "type" (co "pointer type" :av "type" (car ts))))
  (match :v (null vs) nil :do
    (update—push—acontext ac :av "names" (cdr ns) :av "types" (cdr ts)
      :av "values" (cdr vs) :av "block" b)
    (clear—update—eval—acontext ac :instance (car vs))))
```

3.5.3. Декларация функций и методов

Декларация функции вычисляет значение собственно функции, а после сохраняет его в агенте.

```
(aspect "opsem" :context ac :type "function decl" :instance i
  :stage nil :do
    (update-push-actext ac :stage "decl")
    (clear-update-eval-actext ac :instance (aget i "value")))

(aspect "opsem" :context ac :type "function decl" :instance i
  :stage "decl" :value v :agent a :do
    (aset a "function value" (aget i "name") v))
```

Декларация метода происходит аналогичным образом.

```
(aspect "opsem" :context ac :type "method decl" :instance i
  :stage nil :do
    (update-push-actext ac :stage "decl")
    (clear-update-eval-actext ac :instance (aget i "value")))

(aspect "opsem" :context ac :type "method decl" :instance i
  :stage "decl" :value v :agent a :do
    (aset a "method value" (aget i "value" "type" "receiver type")
      (aget i "name") v))
```

3.6. Операторы

Под оператором в данной терминологии понимается инструкция языка, не возвращающая после вычисления никакого значения.

Стоит сразу отметить, что для упрощения операционной семантики все инициализирующие операторы до вычисления выносятся за строку до включающего оператора, вместе с которым образуют отдельный блок.

Для наглядности, сразу приведен пример такого преобразования. На вход подается сле-

дующий фрагмент кода:

```
1 for i := 0; i < 9; i++ { body(i) }
```

Но до операционной семантики доходит уже код, получаемый из следующего фрагмента:

```
1 {
2   i := 0
3   for ; i < 9; i++ { body(i) }
4 }
```

Таким образом, поведение программы не меняется. Такой оператор может встречаться в условных операторах `if`, циклах `for`, операторах выбора `switch`.

3.6.1. Метка

В вычисление помеченного оператора передается блок для корректной декларации переменных, и имя метки для операторов передачи управления.

```
(aspect "opsem" :context ac :type "label" :instance i :do
  (clear—update—eval—acontext ac :instance (aget i "statement")
    :av "block" (aget ac "block") :av "label" (aget i "name")))
```

3.6.2. Пустой оператор

Ничего не делает.

```
(aspect "opsem" :context ac :type "empty stmt")
```

3.6.3. Оператор инкремента

Увеличивает на 1 значение ячейки памяти, на которую действует.

```
(aspect "opsem" :context ac :type "1++ stmt" :instance i :stage nil
:do
  (update—push—acontext ac :stage "apply"))
```

[illegible]

3.6.4. Оператор декремента

Уменьшает на 1 значение ячейки памяти, на которую действует.

```
(aspect "opsem" :context ac :type "1— stmt" :instance i :stage nil
:do
  (update—push—acontext ac :stage "apply")
  (clear—update—eval—acontext ac :aspect "opsem::lvalue"
    :instance (aget i 1)))

(aspect "opsem" :context ac :type "1— stmt" :instance i :stage "apply"
:value v :do
  (aset v "value" (convert-int-to-type (aget i "type")
    (1— (aget v "value"))))))
```

3.6.5. Операторы присваивания

Присваивание сначала вычисляет левую часть (lvalue), затем правую (rvalue). Причем неявное приведение типов не допускается.

```
(aspect "opsem" :context ac :type "assignment stmt" :instance i
  :stage nil :do
    (update—push—acontext ac :stage "eval 2")
    (clear—update—eval—acontext ac :aspect "opsem::lvalue"
      :instance (aget i 1)))
```

```
(aspect "opsem" :context ac :type "assignment stmt" :instance i
  :stage "eval 2" :value v :do
    (update-push-acontext ac :stage "apply" :ac 1 v)
    (clear-update-eval-acontext ac :aspect "opsem::rvalue"
      :instance (aget i 2)))

(aspect "opsem" :context ac :type "1=2" :stage "apply" :ap ac 1 f
  :value s :do (aset f "value" s))
```

Составные присваивания (например, `+=`) выполняют операцию перед записью. Допускается использовать все операции, что определялись в бинарных выражениях.

```
(aspect "opsem" :context ac :type "1+=2" :stage "apply"
  :ap ac 1 f :value s :do
  (aset f "value" (convert-int-to-type (aget f "type")
    (+ (aget f "value") (aget s "value")))))

(aspect "opsem" :context ac :type "1-=2" :stage "apply"
  :ap ac 1 f :value s :do
  (aset f "value" (convert-int-to-type (aget f "type")
    (- (aget f "value") (aget s "value")))))

(aspect "opsem" :context ac :type "1*=2" :stage "apply"
  :ap ac 1 f :value s :do
  (aset f "value" (convert-int-to-type (aget f "type")
    (* (aget f "value") (aget s "value")))))

(aspect "opsem" :context ac :type "1/=2" :stage "apply"
  :ap ac 1 f :value s :do
  (aset f "value" (convert-int-to-type (aget f "type")
    (/ (aget f "value") (aget s "value")))))
```

```
(aspect "opsem" :context ac :type "1<=2" :stage "apply"
  :ap ac 1 f :value s :do
    (aset f "value" (convert-int-to-type (aget f "type")
      (ash (aget f "value") (aget s "value"))))))
```

```
(aspect "opsem" :context ac :type "1>=2" :stage "apply"
  :ap ac 1 f :value s :do
    (aset f "value" (convert-int-to-type (aget f "type")
      (ash (aget f "value") (- (aget s "value"))))))
```

```
(aspect "opsem" :context ac :type "1^=2" :stage "apply"
  :ap ac 1 f :value s :do
    (aset f "value" (convert-int-to-type (aget f "type")
      (logxor (aget f "value") (aget s "value"))))))
```

```
(aspect "opsem" :context ac :type "1|=2" :stage "apply"
  :ap ac 1 f :value s :do
    (aset f "value" (convert-int-to-type (aget f "type")
      (logior (aget f "value") (aget s "value"))))))
```

```
(aspect "opsem" :context ac :type "1%=2" :stage "apply"
  :ap ac 1 f :value s :do
    (aset f "value" (convert-int-to-type (aget f "type")
      (rem (aget f "value") (aget s "value"))))))
```

```
(aspect "opsem" :context ac :type "1&=2" :stage "apply"
  :ap ac 1 f :value s :do
    (aset f "value" (convert-int-to-type (aget f "type")
      (logand (aget f "value") (aget s "value"))))))
```

```
(aspect "opsem" :context ac :type "1&^=2" :stage "apply"
```

```

:ap ac 1 f :value s :do
(aset f "value" (convert-int-to-type (aget f "type")
(logand (aget f "value") (lognot (aget s "value"))))))

```

3.6.6. Условный оператор

Условный оператор `if`.

```

(aspect "opsem" :context ac :type "if stmt" :instance i :stage nil :do
(update-push-acontext ac :stage "apply")
(clear-update-eval-acontext ac :instance (aget i "condition")
:aspect "opsem::rvalue"))

(aspect "opsem" :context ac :type "if stmt" :instance i :stage "apply"
:value c :do
(clear-update-eval-acontext ac :instance (aget i (if c "then"
"else"))))

```

3.6.7. Оператор выбора

Вычисление оператора выбора `switch` начинается с вычисления контролирующего выражения - на равенство которому проверяются условия веток.

```

(aspect "opsem" :context ac :type "expr switch stmt" :instance i
:stage nil :do
(update-push-acontext ac :stage "apply cont"
:av "block" (aget ac "block"))
(clear-update-eval-acontext ac :instance (aget i "controlled")
:aspect "opsem::rvalue"))

```

Вычисляем условие первой ветки, запускаем вычисление для последующих веток. Если контролирующего выражения нет, то считается, что условия веток проверяются на логическую истину `true`.

```
(aspect "opsem" :context ac :type "expr switch stmt" :instance i
  :stage "apply cont" :value cont :ap i "cases" cs
  :v (null cs) nil :ap ac "block" b :do

  (match :v cont nil :p cont T :do
    (update-push-acontext ac :stage "evaluating cases" :av "cont" cont
      :av "left" (cdr cs) :av "block" b)
    (clear-update-eval-acontext ac :instance (car cs) :av "cont" cont)
      :av "block" b))
```

После вычисления `expr case clause` возвращает истину, если хотя бы одно условие пошло. То есть эта стадия работает до тех пор, пока не сработала предыдущая ветка, или пока все ветки не закончились.

```
(aspect "opsem" :context ac :type "expr switch stmt" :instance i
  :stage "evaluating cases" :ap ac "cont" cont :ap ac "left" left
  :v (null left) nil :value v :v v nil :ap ac "block" b :do

  (update-push-acontext ac :av "cont" cont :av "left" (cdr left)
    :av "block" b)
  (clear-update-eval-acontext ac :instance (car left) :av "cont" cont
    :av "block" b))
```

У каждой ветки может быть несколько условий, проверяем каждое из них.

```
(aspect "opsem" :context ac :type "expr case clause" :instance i
  :stage nil :ap ac "cont" cont :ap i "cases" cs :ap ac "block" b :do

  (match :v (null cs) T :do nil
  :exit
  (update-push-acontext ac :stage "evaluating" :av "cont" cont
    :av "left" (cdr cs) :av "block" b))
```

```
(clear-update-eval-acontext ac
  :instance (mo "1==2" :av 1 cont :av 2 (car cs))
  :aspect "opsem::rvalue") :av "block" b))
```

Проверяем, подошел ли предыдущий случай этой ветки. В противном случае запускаем проверку последующих.

```
(aspect "opsem" :context ac :type "expr case clause" :instance i
  :stage "evaluating" :value v :ap ac "cont" cont :ap ac "left" left
  :ap ac "block" b :do

  (match :v v nil
    :do (nmatch :v (null left) T :exit nil
      :v (otype (car left)) "default"
      :exit
      (clear-update-eval-acontext ac :stage "evaluating body"
        :av "left" (aget i "statements") :av "block" b)
      :do
      (update-push-acontext ac :av "cont" cont :av "left" (cdr cs)
        :av "block" b)
      (clear-update-eval-acontext ac :instance (mo "1==2" :av 1 cont
        :av 2 (car cs)) :aspect "opsem::rvalue") :av "block" b)
      :exit (clear-update-eval-acontext ac :stage "evaluating body"
        :av "left" (aget i "statements") :av "block" b))))
```

Выполняется тело первой ветки, условие которой совпало с контролирующим выражением оператора выбора.

```
(aspect "opsem" :context ac :type "expr case clause" :instance i
  :stage "evaluating body" :ap ac "left" left :ap ac "block" b :do
  (match :v (null left) T :do T :exit
    (update-push-acontext ac :av "left" (cdr left) :av "block" b)
    (clear-update-eval-acontext ac :instance (car left) :av "block" b)))
```

3.6.8. Операторы циклов

Цикл с условием выполняет собственное тело до тех пор, пока условие цикла истинно.

Начальная стадия отвечает за вычисление условия цикла.

```
(aspect "opsem" :context ac :type "for condition" :instance i
  :stage nil :do
    (update-push-actext ac :stage "apply")
    (clear-update-eval-actext ac :instance (aget i "condition")
      :aspect "opsem::rvalue"))
```

В случае истинности условия, стадия `apply` выполняет тело цикла. После чего управление переходит к первой стадии.

```
(aspect "opsem" :context ac :type "for condition" :instance i
  :stage "apply" :value v :v v T :do
    (update-push-actext ac :stage nil)
    (clear-update-eval-actext ac :instance (aget i "body")))
```

Классический цикл `for`, помимо определенных в предыдущем случае, имеет оператор, выполнение которого происходит после каждого вычисления тела цикла.

Оператор, выполняющийся один раз до цикла лексически выносится на строку до цикла. Так же происходит со всеми операторами, в которых присутствуют такие инициализирующие операторы.

Сначала вычисляется условие цикла.

```
(aspect "opsem" :context ac :type "for clause" :instance i
  :stage nil :do
    (update-push-actext ac :stage "apply")
    (clear-update-eval-actext ac :instance (aget i "condition")
      :aspect "opsem::rvalue"))
```

Если вычисленное условие истинно, то выполняем тело цикла.

```
(aspect "opsem" :context ac :type "for clause" :instance i
  :stage "apply" :value v :v v T :do
  (update-push-actext ac :stage "post")
  (clear-update-eval-actext ac :instance (aget i "body"))))
```

После чего, выполняем соответствующий оператор, и возвращаемся к началу.

```
(aspect "opsem" :context ac :type "for clause" :instance i
  :stage "post" :do
  (update-push-actext ac :stage nil)
  (clear-update-eval-actext ac :instance (aget i "post"))))
```

Далее пойдет речь о циклах с перебором элементов коллекций. Поскольку при этом переменные принимают лишь копированные значения ячеек, то эти копии необходимо создавать для каждого хранилища. Напишем функцию `copy-cells`, которая создает по списку ячеек, список ячеек, элементы которого имеют такие же значения.

```
(defun copy-cells (lst)
  (let ((r nil)) (dolist (c (reverse lst) r)
    (setq r (cons (mo "cell" :av "type" (aget c "type")
                     :av "value" (aget c "value"))
                  r))))))
```

Начнем с индексированных хранилищ - к элементам которых доступно обращение по индексу.

Сначала вычислим хранилище, элементы которого необходимо перебирать. Для каждого возможного типа хранилища заведем отдельную стадию.

```
(aspect "opsem" :context ac :type "for range indexable" :instance i
  :stage nil :ap i "indexable" ind :do
  (update-push-actext ac :stage (otype (aget ind "type")))
  (clear-update-eval-actext ac :instance ind :aspect "opsem::rvalue"))
```

Случай среза сведем к случаю массива.

```
(aspect "opsem" :context ac :type "for range indexable" :instance i
  :stage "slice type" :value sl :ap sl "offset" ofst
  :ap sl "length" len :do

  (clear—update—eval—acontext ac :instance (mo "for range indexable"
:av "index" (aget i "index")
:av "value" (aget i "value")
:av "operation" (aget i "operation")
:av "indexable" (mo "array value"
  :av "type" (co "array type" :av "len" len
    :ap "elem type" (aget sl "type" "elem type"))
  :av "elements" (copy—cells
    (subseq (aget sl (aseq "array" "elements")) ofst (+ ofst len))))
:av "body" (aget i "body"))))
```

Случай строки так же сведем к случаю массива.

```
(aspect "opsem" :context ac :type "for range indexable" :instance i
  :stage "string type" :value pr :ap pr "value" s :do

  (clear—update—eval—acontext ac :instance (mo "for range indexable"
:av "index" (aget i "index")
:av "value" (aget i "value")
:av "operation" (aget i "operation")
:av "indexable" (mo "array value"
  :av "type" (co "array type" :av "len" (length s)
    :av "elem type" (co "unsigned int type" :av "bit size" 8))
  :av "value" (map 'list #'char—code s))))
```

В случае массива запускаем итерационное вычисление.

```
(aspect "opsem" :context ac :type "for range indexable" :instance i
```

```

:stage "array type" :value arr :do
  (clear—update—eval—acontext ac :stage "evaluating" :av "now" 0
    :av "left" (copy—cells (aget arr "value"))))

```

На стадии `evaluating` выполняется итерация цикла. То есть сначала присваиваются соответствующие значения переменным индекса и ключа, а потом выполняется тело цикла с этими значениями.

```

(aspect "opsem" :context ac :type "for range indexable" :instance i
  :stage "evaluating" :ap ac "left" left :av ac "now" k :agent a :do

  (aset a "variable cell" (aget i "index") k)
  (aset a "variable cell" (aget i "value") (car left))
  (update—push—acontext ac :av "now" (1+ k) :av "left" (cdr left))
  (clear—update—eval—acontext ac :instance (aget i "body")))

```

Теперь рассмотрим случай хранилища - карты.

Сначала вычисляем значение карты.

```

(aspect "opsem" :context ac :type "for range map" :instance i
  :stage nil :do
  (update—push—acontext ac :stage "save")
  (clear—update—eval—acontext ac :instance (aget i "map")
    :aspect "opsem::rvalue"))

```

Затем сохраняем значения затеняемых переменных.

```

(aspect "opsem" :context ac :type "for range map" :instance i
  :stage "save" :value m :ap m "entry" mv
  :p (attributes mv) ks :p nil vs :do

  (dolist (k (reverse ks)) (setq vs (cons (aget mv k) vs)))
  (clear—update—eval—acontext ac :stage "evaluating" :av "keys" ks

```

```
:av "values" (copy-cells vs)))
```

На стадии `evaluating` выполняется итерация цикла. То есть сначала присваиваются соответствующие значения переменным индекса и ключа, а потом выполняется тело цикла с этими значениями.

```
(aspect "opsem" :context ac :type "for range map" :instance i
  :stage "evaluating" :ap ac "keys" ks :ap ac "values" vs :agent a :do

  (aset a "variable cell" (aget i "key") (car ks))
  (aset a "variable cell" (aget i "values") (car vs))
  (update-push-acontext ac :av "keys" (cdr ks) :av "values" (cdr vs))
  (clear-update-eval-acontext ac :instance (aget i "body")))
```

Наконец, рассмотрим цикл с перебором элементов по целому числу. Этот хоть и является урезанной версией классического цикла, но имеет собственный синтаксис.

Сначала вычисляется количество итераций. Переменная значений будет принимать значения от 0 включительно, до вычисленного значения исключительно.

```
(aspect "opsem" :context ac :type "for range int" :instance i
  :stage nil :do
  (update-push-acontext ac :stage "start")
  (clear-update-eval-acontext ac :instance (aget i "max")
    :aspect "opsem::rvalue"))
```

На стадии `start` начинается итерационное вычисление тела цикла.

```
(aspect "opsem" :context ac :type "for range int" :instance i
  :stage "start" :value m :do
  (clear-update-eval-acontext ac :stage "evaluating" :av "max" m
    :av "now" 0))
```

На стадии `evaluating` выполняется итерация цикла. То есть сначала присваивается со-

ответствующее значение переменной значения, а потом выполняется тело цикла с этими значениями.

```
(aspect "opsem" :context ac :type "for range int" :instance i
  :stage "evaluating" :ap ac "max" m :av ac "now" k
  :agent a :v (< k m) T :do

  (aset a "variable cell" (aget i "value") k)
  (update-push-acontext ac :av "max" m :av "now" (1+ k))
  (clear-update-eval-acontext ac :instance (aget i "body")))
```

3.6.9. Оператор return

Вычисление оператора `return` начинается с вычисления возвращаемых значений (если есть).

```
(aspect "opsem" :context ac :type "return stmt" :instance i
  :stage nil :ap ac 1 left :do

  (update-push-acontext ac :stage "exiting before function")
  (match :v (null left) T :exit
    (update-push-acontext ac :stage "evaluating"
      :av "left" (cdr left))
    (clear-update-eval-acontext ac :instance (car left)
      :aspect "opsem::rvalue"))))

(aspect "opsem" :context ac :type "return stmt" :instance i
  :stage "evaluating" :value v :ap ac "left" left
  :ap ac "done" d :do

  (match :v (null left) T
    :do (reverse d)
    :exit
```

```
(update-push-acontext ac :av "left" (cdr left) :av "done" (cons v d))
(clear-update-eval-acontext ac :instance (car left)
  :aspect "opsem::rvalue"))))
```

Затем выходим из всех ранее принятых к вычислению конструкций, начиная с тех, что начали вычисляться последними, до тех пор, пока не встретим функцию. В этом случае возвращаем вычисленные значения, и заканчивает вычисление найденной функции. Причем при встрече с блоком не забываем возвращать прежние значения затененным переменным.

```
(aspect "opsem" :context ac :type "return stmt" :instance i :stage
  "exiting before function"
  :value d :p (pop-acontext ac) ac1 :p (aget ac1 "instance") i1 :do
  (nmatch :v (is-instance i1 "block") T
    :exit (push-acontext ac) (update-eval-acontext ac1 :stage
      "variable back"))
  :v (is-instance i1 "function call") T
  :exit d (push-acontext ac1)
  :do d (eval-acontext ac)
  )
```

3.6.10. Оператор break

Вычисление оператора `break` происходит похожим с предыдущим случаем образом. Только вместо завершения функции, данный оператор прерывает цикл. Такие операторы называют операторами передачи управления.

```
(aspect "opsem" :context ac :type "break stmt" :instance i
  :stage nil :ap i 1 ln :p (pop-acontext ac) ac1 :ap ac1 "instance" i1
  :ap ac1 "label" ln1 :do

  (nmatch
    :v (is-instance i1 "block") T
```

```

:exit
(push-acontext ac) (update-eval-acontext ac1 :stage "variable back")
:v (and (is-instance i1 "for stmt") (or (not ln) (string= ln ln1)))
:exit nil
:v (and (is-instance i1 "switch stmt") (or (not ln) (string= ln ln1)))
:exit nil
:do (eval-acontext ac)))

```

3.6.11. Оператор continue

Вычисление оператора `continue` происходит аналогичным образом.

```

(aspect "opsem" :context ac :type "continue stmt" :instance i
  :stage nil :ap i 1 ln :p (pop-acontext ac) ac1 :ap ac1 "instance" i1
  :ap ac1 "label" ln1 :do

  (nmatch
    :v (is-instance i1 "block") T
    :exit
    (push-acontext ac) (update-eval-acontext ac1 :stage "variable back")
    :v (and (is-instance i1 "for stmt") (or (not ln) (string= ln ln1)))
    :exit nil
    :do (eval-acontext ac)))

```

3.6.12. Оператор goto

Вычисление оператора `goto` происходит аналогичным образом.

```

(aspect "opsem" :context ac :type "goto stmt" :instance i
  :stage nil :ap i 1 ln :p (pop-acontext ac) ac1
  :ap ac1 "instance" i1 :do

  (match :ap i1 "label" ln1 :v ln1 T :v (string= ln ln1) T

```

```
:do (update-eval-actext ac1 :stage nil)
:exit (match :v (is-instance i1 "block") T
  :ap i1 "label positions" lp :ap lp ln pos
  :do (match :v pos T :ap ac1 "statements" sts
    :p (nthcdr pos sts) scdr :do
      (update-push-actext ac1 :av "left" (cdr scdr))
      (update-push-actext ac1 :instance (car scdr))
      (eval-actext ac))
    :exit
    (push-actext ac) (update-push-actext ac1 :stage "variable
back"))
:exit nil)))
```